Backend Report

Project Description

For my final project, I am proposing to develop a database to mimic an online storefront/library management service. This project is modeled after other platforms like Steam or the Epic Games Store. The system will store information about users, games, the games played by users, and developers of the games.

Explanation of Problem

Online commercial platforms are becoming increasingly common in everyday life. Online platforms must be able to process transactions frequently and allow many users to utilize services at once. Without a properly structured database, records could become corrupted with duplicate information, missing information, or system breaking errors.

ER Diagram

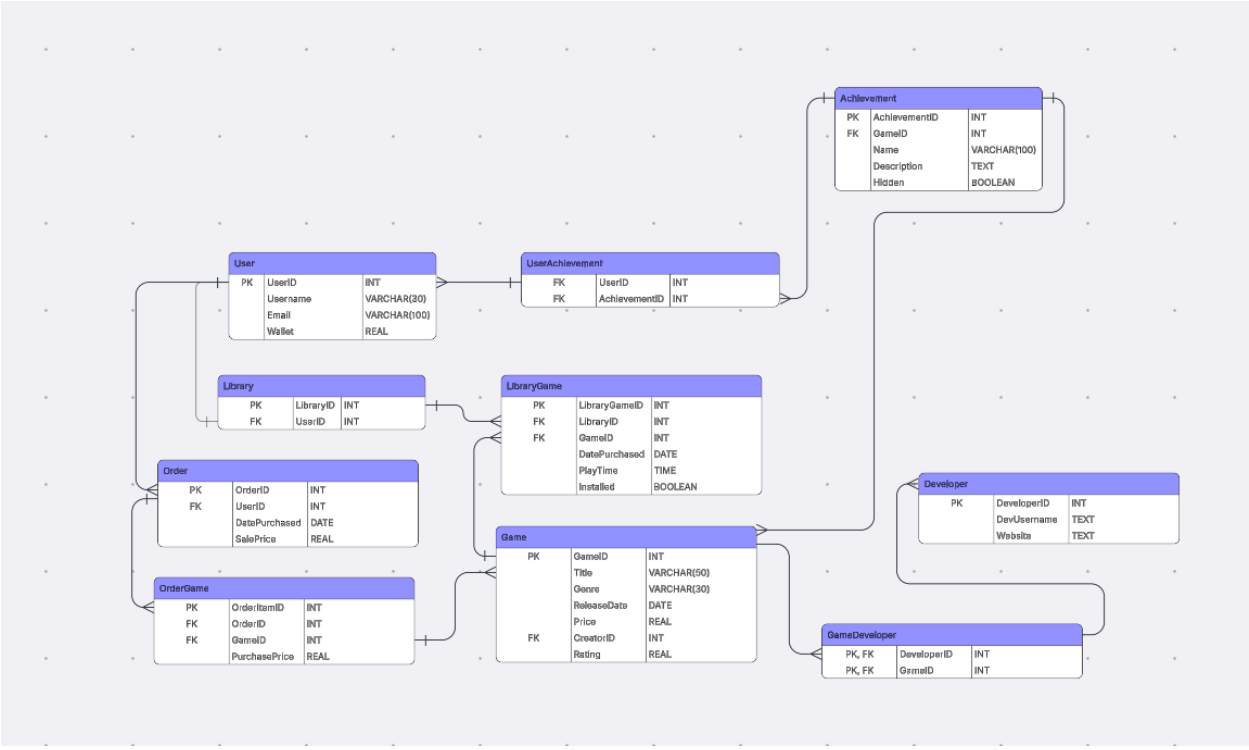


Figure 1: Entity Relationship Diagram for the Database

Tables and Relationships

Users

The User table stores information about the general consumers of the platform. Users have an ID, username, email address, and account balance. Each user can have one library that tracks all games they have purchased. A user may place one order at a time, but orders can contain multiple games to purchase.

Library

The Library table represents a user's purchased games repository. Each user has one library that contains all games they own and statistics on the games.

LibraryGame

The LibraryGame table is a junction table between Library and Games. This table is necessary to facilitate the many-to-many relationship between Library and Games, as each library can have every game.

Games

The Games table stores information about games available on the system to purchase. Each game may appear in multiple libraries and orders, and can have multiple developers and achievements.

Developers

The Developers table stores information on the developers of games.

GameDeveloper

The GameDeveloper table is a junction table between Games and Developers. A developer can work on multiple games and one game can have multiple developers.

Orders

The Orders table stores information about purchases and transactions made by users. A user can place multiple orders, but each order belongs to one user.

OrderGame

The OrderGames table is a junction table between Orders and Games. An order can contain multiple games and allows a game to appear in multiple different orders.

Achievements

The Achievements table stores achievements associated with games. A game can have multiple achievements.

UserAchievement

UserAchievement records which achievements have been earned by users. Because a user can earn many achievements and each achievement can be obtained by many users, a junction table is needed.

Primary Keys

The primary keys of each table are the unique ID numbers for the entities of the table, such as:

- Users.UserID
- Library.LibraryID
- Games.GameID
- Developers.DeveloperID
- Orders.OrderID
- Achievements.AchievementID

Foreign Keys

The foreign keys establish the relationships between the tables, most notably for the junction tables.

- LibraryGame uses LibraryID and GameID.
- OrderGame uses OrderID and GameID.
- GameDeveloper uses GameID and DeveloperID.
- UserAchievement uses UserID and AchievementID.

Normalization

This database was designed in the format of Third Normal Form. Third Normal Form was the best decision because this form supports many to many relationships while also limiting redundancies of data across entries and tables.

Views

```
CREATE VIEW GameDeveloperView AS
SELECT
    Games.GameID,
    Games.Title,
    Developers.DevUsername AS Developer,
    Games.Genre,
    Games.Price,
    Games.Rating
```

```
FROM Games
INNER JOIN GameDeveloper
    ON Games.GameID = GameDeveloper.GameID
INNER JOIN Developers
    ON GameDeveloper.DeveloperID = Developers.DeveloperID;

SELECT * FROM GameDeveloperView WHERE Rating >= 8.0;
```

Figure 2. View from queries.

For the project, I created a view for displaying information relevant to a developer. This view allows for products to be compared by developers based on the games they've worked on.

Indexes

```
CREATE INDEX idx_games_title
    ON Games(Title);

CREATE INDEX idx_orders_userID
    ON Orders(UserId);

CREATE INDEX idx_librarygame_gameID
    ON LibraryGame(GameID);
```

For the indices, I decided to go with 3 elements that are often referenced, as most users are players so being able to reference their games by title, orders, and libraries need to occur quickly.

Queries

Query 1: Display a User's Library

```
SELECT
    Users.Username,
    Library.LibraryID
FROM Users
LEFT JOIN Library
    ON Users.UserID = Library.UserID;
```

This query identifies the library associated with each user. The LEFT JOIN ensures that users are displayed even if they do not have a library. This is important for finding records before making a transaction.

Query 2: Find All Games made by Developers

```
SELECT
    Games.GameID,
    Games.Title,
    Developers.DevUsername AS Developer,
    Games.Genre,
    Games.Price,
    Games.Rating
FROM Games
INNER JOIN GameDeveloper
    ON Games.GameID = GameDeveloper.GameID
INNER JOIN Developers
    ON GameDeveloper.DeveloperID = Developers.DeveloperID;
```

This query retrieves all games associated with a specific developer. It demonstrates the use of a junction table to facilitate many-to-many relationships.

Query 3: Find all Orders placed by a User.

```
SELECT
    Users.Username,
    Orders.OrderID,
    Orders.DatePurchased,
    Orders.SalePrice
FROM Users
INNER JOIN Orders
    ON Users.UserId = Orders.UserID;
```

This query retrieves the order history for a specific user. Finding orders can prevent users from accidentally purchasing the same games multiple times, since users only need one copy per account.

Query 4 Reduce funds for purchase

```
UPDATE Users
            SET Wallet = Wallet - ?
            WHERE UserID = ?
```

This query deducts the cost of a purchase from a user's wallet. It demonstrates how the database can modify existing records based on a user's transaction and can be used to actually make transactions for purchases. This is an essential query for the database.

Script-Based Front End

The database will be accessed through a script-based frontend that provides a user friendly interface to access the database. A simple Python script was developed to allow users to sign in and: Purchase games, view their friends' libraries, and view the storefront. The script sends SQL queries to the database based on the options selected in the menu. Currently the system supports 6 applications: Viewing the storefront, purchasing a game, viewing libraries, viewing developer information, and getting a user's current balance. The database handles the storage, relationships, and queries, while the frontend provides the interface. To expand functionality, the tool could be developed to:

- Track User Logins
- View past orders
- Check games on sale
- Have a designated friends list